

Predicting 30-Day Hospital Readmission in Diabetic Patients (Part 2)

Week 2 Project Report

Prepared by: Peter Adepoju

Email: peter.o.adepoju@gmail.com

Dataset: UCI Diabetes 130-US Hospitals

Date: June 12, 2026

Contents

1	Exploratory Data Analysis	3
1.1	The EDA Workflow	3
1.2	Class Balance Analysis	3
1.3	Descriptive Statistics of Clinical Features	4
1.4	Distribution Shapes and Outlier Analysis	4
1.5	Bivariate Analysis: Numeric Features vs. Target	5
1.6	Bivariate Analysis: Categorical Features vs. Target	6
1.7	Readmission Rate by Age Group	7
1.8	Feature–Target Correlation Analysis	8
1.9	Correlation Between Features	10
1.10	Key EDA Findings and Their Modelling Implications	11
2	Data Leakage and Splitting Strategy	12
2.1	Patient-Level Leakage	12
2.2	GroupShuffleSplit: The Solution	13
2.3	The 70/15/15 Split Rationale	14
2.4	Leakage Verification	14
2.5	The Preprocessing Pipeline	15
2.6	SMOTE	15
3	Baseline Models	17
3.1	The Epistemological Role of Baselines	17
3.2	The Dummy Classifier	17
3.3	Logistic Regression	18
3.4	Baseline Results Summary	20
4	Advanced Ensemble Models	21
4.1	The Case for Ensemble Methods on This Dataset	21
4.2	Common Training Setup	21
4.3	Random Forest	22

4.3.1	Hyperparameter Choices	22
4.3.2	Validation Performance	23
4.4	Gradient Boosting	23
4.4.1	Hyperparameter Choices	23
4.4.2	Validation Performance	23
4.5	XGBoost	24
4.5.1	Algorithmic Innovations	24
4.5.2	Hyperparameter Choices	24
4.5.3	Validation Performance	25
4.6	Validation Set Model Comparison	25
4.7	Final Retraining on Train+Val Combined	25
References		26

Chapter 1

Exploratory Data Analysis

“Exploratory data analysis can never be the whole story, but nothing else can serve as the foundation stone.”

— John W. Tukey

Exploratory Data Analysis (EDA) is the practice of interrogating data before modelling to build an honest understanding of its structure, distributions, and relationships. In this chapter we report both what we found and what those findings imply for the modelling choices in subsequent chapters.

1.1. The EDA Workflow

Our EDA is structured in two notebooks: univariate analysis and bivariate analysis. Univariate EDA examines each feature in isolation; bivariate EDA examines each feature’s relationship with the target variable. Both are prerequisites for informed feature engineering and model selection.

1.2. Class Balance Analysis

The working cohort of 69,987 first encounters contains **62,255 negatives** (88.8%) and **7,732 positives** (11.2%). This 1:7.9 imbalance ratio is the most important data characteristic for the modelling pipeline.

Key result. The class imbalance ratio of 1:7.9 mandates the use of imbalance-aware evaluation metrics (ROC-AUC, Average Precision, Recall) and imbalance-handling techniques (SMOTE, class weighting). Accuracy is reported for completeness only and should not be used for model comparison.

1.3. Descriptive Statistics of Clinical Features

Table 1.1 presents summary statistics for the core clinical features after cleaning and before feature engineering transforms.

Table 1.1. Descriptive statistics for clinical variables in the 69,987-encounter working cohort. The large standard deviations and wide max values for prior visit counts confirm heavy right skew, motivating log-transformation in the preprocessing pipeline.

Feature	Mean	Std	Min	Q1	Median	Max
Time in hospital (days)	4.29	2.94	1	2	3	14
Lab procedures	43.0	19.8	1	31	44	132
Procedures performed	1.40	1.74	0	0	1	6
Medications count	15.75	8.25	1	10	14	81
Distinct diagnoses	7.27	1.99	1	6	8	16
Prior outpatient visits	0.30	1.11	0	0	0	42
Prior emergency visits	0.12	0.63	0	0	0	54
Prior inpatient visits	0.33	0.82	0	0	0	17
Total prior visits	0.75	1.55	0	0	0	58

Three observations stand out. First, `number_diagnoses` has a mean of 7.27, these patients carry, on average, seven distinct diagnoses. This is a population of multiply-comorbid patients for whom simple risk scores are likely to be inadequate. Second, the prior visit counts (`number_outpatient`, `number_emergency`, `number_inpatient`) all have medians of 0 but means above 0, indicating right-skewed distributions where most patients have no prior visits but a minority have many. Third, `num_medications` has a mean of 15.75 and a maximum of 81, some patients are on more than 80 medications simultaneously, an extraordinary pharmacological complexity.

1.4. Distribution Shapes and Outlier Analysis

Table 1.2. IQR-based outlier counts and percentages for continuous clinical features. Values below $Q_1 - 1.5 \times \text{IQR}$ or above $Q_3 + 1.5 \times \text{IQR}$ are classified as outliers.

Feature	Lower Bound	Upper Bound	Outlier Count	Outlier %
Time in hospital	-4.0	12.0	1,430	2.04%
Lab procedures	-8.0	96.0	101	0.14%
Procedures performed	-3.0	5.0	3,725	5.32%
Medications count	-5.0	35.0	1,882	2.69%
Distinct diagnoses	1.5	13.5	231	0.33%
Prior outpatient	0.0	0.0	9,762	13.95%
Prior emergency	0.0	0.0	5,781	8.26%
Prior inpatient	0.0	0.0	14,764	21.10%
Total prior visits	-1.5	2.5	6,623	9.46%

1.5. Bivariate Analysis: Numeric Features vs. Target

For each continuous feature, we test whether its distribution differs significantly between readmitted ($y = 1$) and non-readmitted ($y = 0$) patients using the **Mann-Whitney U test** (Mann & Whitney, 1947).¹

The Mann-Whitney test statistic U tests the null hypothesis that the two distributions are identical:

$$H_0 : P(X > Y) = 0.5 \quad \text{vs.} \quad H_1 : P(X > Y) \neq 0.5 \quad (1.1)$$

Table 1.3 presents the results for all 12 continuous features.

Table 1.3. Mann-Whitney U test results comparing the distribution of each clinical feature between readmitted and non-readmitted patients. Eleven of twelve features show statistically significant differences at $p < 0.05$ after Bonferroni correction ($\alpha_{\text{adjusted}} = 0.05/12 \approx 0.0042$). All p -values except `num_procedures` fall below 0.001.

Feature	Mean (No Readmit)	Mean (Readmit)	Difference	Significant?
Time in hospital	4.245	4.841	+0.595	Yes
Lab procedures	42.78	45.76	+2.975	Yes
Procedures performed	1.407	1.378	-0.029	No
Medications count	15.65	16.90	+1.250	Yes
Distinct diagnoses	7.238	7.655	+0.417	Yes
Prior inpatient visits	0.297	0.760	+0.463	Yes
Prior emergency visits	0.118	0.211	+0.093	Yes
Prior outpatient visits	0.298	0.385	+0.087	Yes
Total prior visits	0.713	1.357	+0.644	Yes
Active medications	1.184	1.215	+0.031	Yes
Medication changes	0.261	0.311	+0.050	Yes
Age (ordinal)	6.033	6.324	+0.291	Yes

The one non-significant feature, `num_procedures`, has near-identical means (1.407 vs. 1.378, $p = 0.91$). The strongest differences are for **total prior visits** (0.713 vs. 1.357, $\Delta = 0.644$) and **prior inpatient visits** (0.297 vs. 0.760, $\Delta = 0.463$), both highly significant ($p < 0.0001$). This confirms prior hospitalisation as the dominant signal in the feature set, consistent with the literature.

¹The Mann-Whitney U test is the non-parametric equivalent of the two-sample t -test. It does not assume normality, making it appropriate for the skewed distributions in this dataset. A clear tutorial is at <https://www.statmodels.org/stable/generated/statmodels.stats.weightstats.mannwhitneyu.html>.

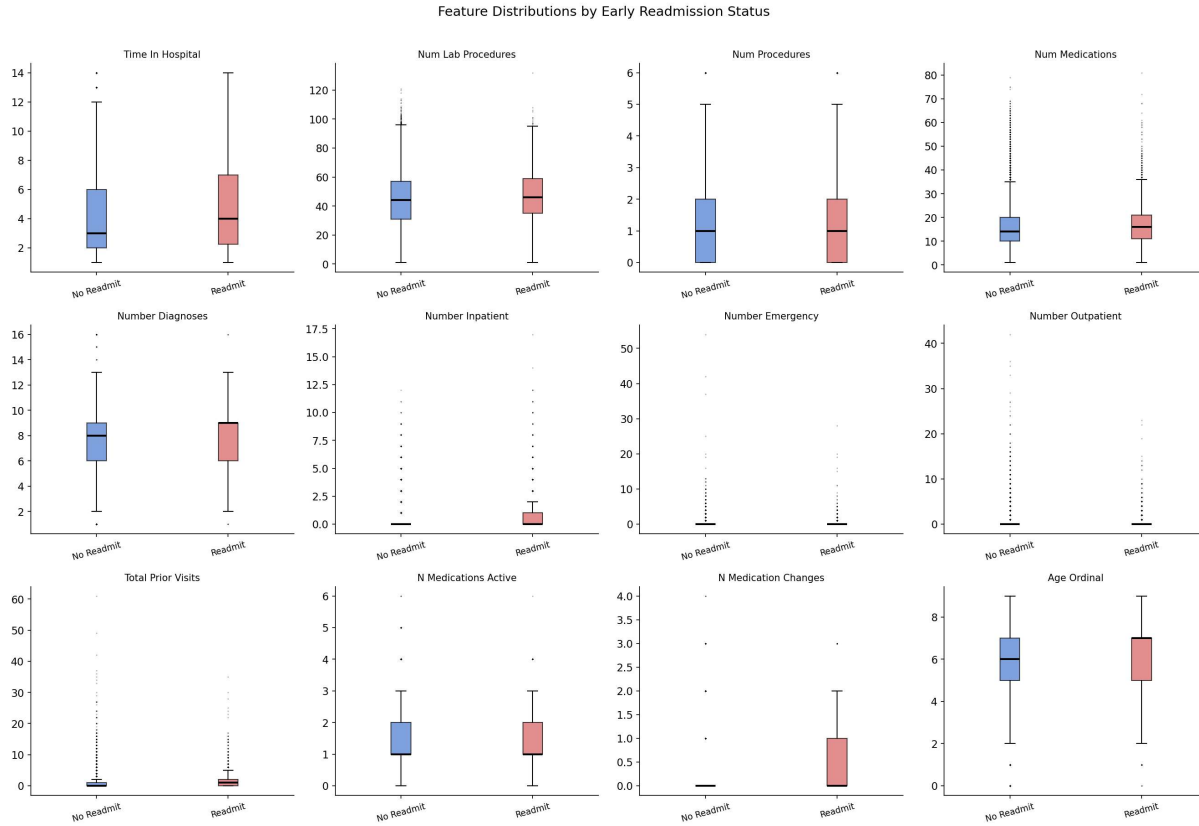


Figure 1.1. Box plots of clinical features stratified by readmission label. The consistently higher medians and upper quartiles for readmitted patients (right boxes) confirm the Mann-Whitney U test results quantitatively. Note the long right tails on the prior visit count features, which motivate log-transformation in the preprocessing pipeline.

1.6. Bivariate Analysis: Categorical Features vs. Target

For categorical features, we use the **Pearson chi-squared test** (Pearson, 1900) to test for independence between the feature and the target:

$$\chi^2 = \sum_{i=1}^r \sum_{j=1}^c \frac{(O_{ij} - E_{ij})^2}{E_{ij}} \quad (1.2)$$

where O_{ij} is the observed cell count and E_{ij} is the expected count under independence. Table 1.4 reports results for the key categorical predictors.²

²The chi-squared test for independence is a standard tool in contingency table analysis. See <https://www.statstutor.ac.uk/resources/uploaded/chi-squaredtest.pdf> for an accessible treatment with clinical examples.

Table 1.4. Pearson chi-squared test of independence between categorical features and the binary readmission target. All features with $p < 0.05$ show a statistically significant association with readmission.

Feature	Categories	χ^2 Statistic	Significant?
diag_1_cat	10	88.50	Yes
on_diabetes_med	2	45.47	Yes
admission_type_id	8	46.80	Yes
had_prior_inpatient	2	1009.69	Yes (strongest)
A1C_tested	2	18.22	Yes
med_changed	2	16.49	Yes
race	5	29.54	Yes
glucose_tested	2	6.01	Yes
primary_diag_is_diabetes	2	2.08	No

The `had_prior_inpatient` binary flag has by far the largest chi-squared statistic ($\chi^2 = 1009.69$), reinforcing the finding from the numeric analysis that prior hospitalisation is the dominant predictor. Interestingly, `primary_diag_is_diabetes` is not significantly associated with readmission ($p = 0.15$), suggesting that whether diabetes is listed as the primary diagnosis provides little additional predictive information beyond the other features in the model.

1.7. Readmission Rate by Age Group

Figure 1.2 shows the readmission rate by decade age bracket. The relationship is broadly monotonically increasing with age, rising from approximately 8% in the 30–40 cohort to about 11% in the 80–90 cohort. However, the relationship is not strictly monotonic and the absolute differences between age groups are modest.

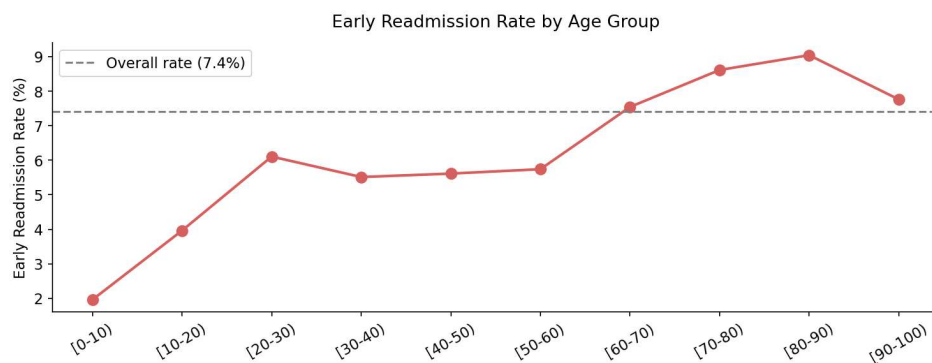


Figure 1.2. 30-day readmission rate by decade age bracket. Older patients have higher readmission rates, but the gradient is modest (range approximately 7–12%). The youngest age groups (<30) have very small sample sizes, making their readmission rate estimates unreliable (wide confidence intervals). The elderly cohort (80–90) represents approximately 20% of the total population but contributes disproportionately to readmission burden.

1.8. Feature–Target Correlation Analysis

Point-biserial correlations between each continuous feature and the binary target are presented in Figure 1.3. The top five predictors by absolute correlation are:

1. `number_inpatient`: $r = 0.149$
2. `had_prior_inpatient`: $r = 0.120$
3. `total_prior_visits`: $r = 0.101$
4. `discharge_disposition_id`: $r = 0.058$
5. `number_diagnoses`: $r = 0.055$

The highest correlation, $r = 0.149$, is modest. The entire feature set has weak individual linear correlations with the target. This is not surprising: 30-day readmission in diabetic patients is a multifactorial outcome determined by dozens of interacting clinical, social, and behavioural factors. No single feature should be expected to predict it strongly on its own. This pattern — many weak predictors, no dominant single predictor — is exactly the setting where ensemble methods, which aggregate many weak signals, should outperform simpler linear models.

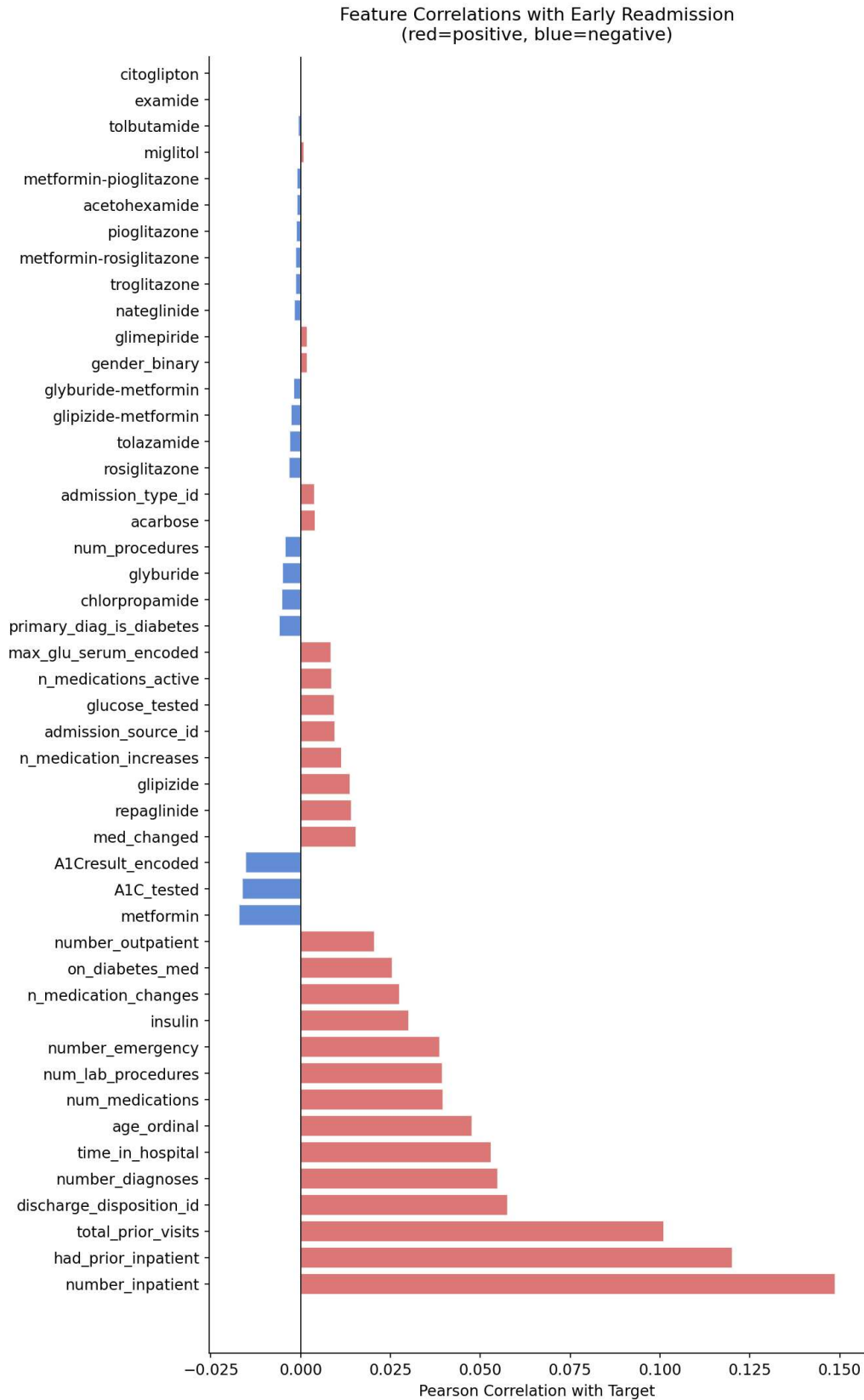


Figure 1.3. Point-biserial correlations between engineered features and the binary readmission target, sorted by absolute magnitude. Prior hospitalisation features (`number_inpatient`, `had_prior_inpatient`, `total_prior_visits`) are the dominant signal. The modest magnitude of all correlations ($|r| < 0.15$) indicates that no single feature is strongly predictive, motivating ensemble approaches that can aggregate many weak signals.

1.9. Correlation Between Features

Figure 1.4 shows the Pearson correlation matrix among the continuous engineered features. The main collinearities to note are:

- `number_inpatient` and `had_prior_inpatient`: $r \approx 0.83$ (by construction, since the binary flag is derived from the count).
- `total_prior_visits` and `number_inpatient`: $r \approx 0.73$ (the inpatient component dominates the aggregate).
- `num_medications` and `n_medications_active`: $r \approx 0.75$ (both measure medication burden).

For tree-based models (Random Forest, Gradient Boosting, XGBoost), high collinearity between predictors is largely harmless: the models select among correlated predictors based on information gain and are not sensitive to linear dependence in the feature matrix. For Logistic Regression with L2 regularisation, collinearity can inflate coefficient standard errors but does not bias the predictions. We therefore retain all features and allow the model to handle collinearity implicitly.

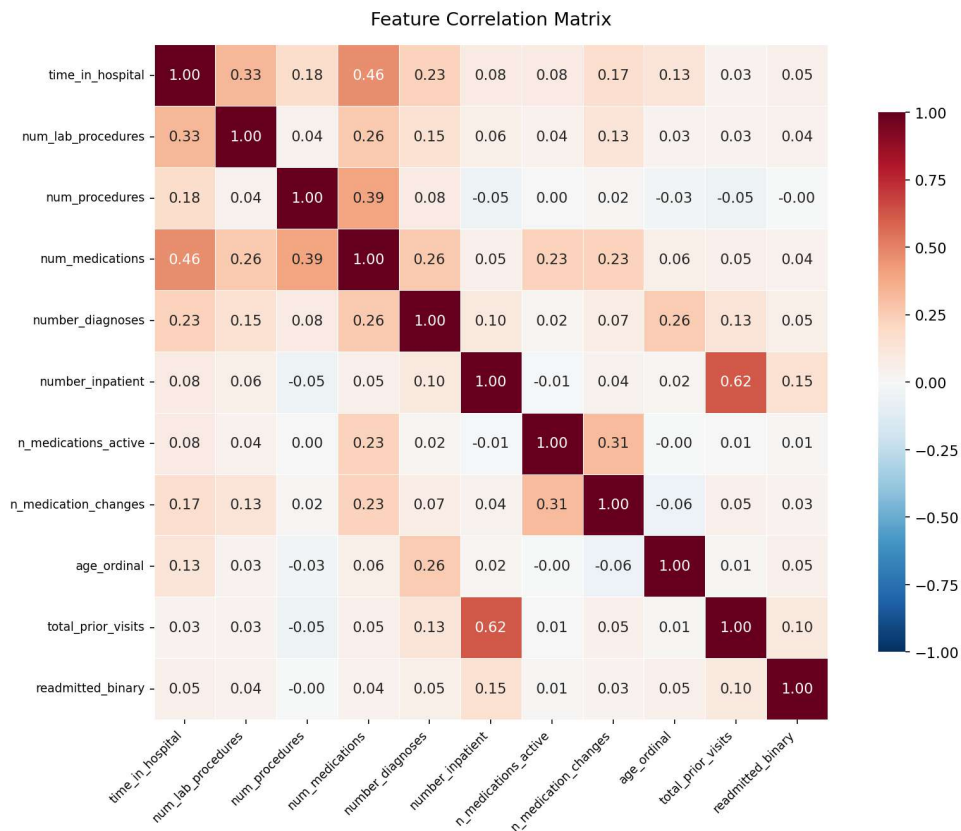


Figure 1.4. Pearson correlation matrix for the continuous engineered features. The strongest inter-feature correlations are between prior visit aggregates (`number_inpatient` vs. `total_prior_visits`, $r \approx 0.73$) and between medication count features (`num_medications` vs. `n_medications_active`, $r \approx 0.75$). These collinearities are tolerated in tree-based models but noted for interpretation of Logistic Regression coefficients.

1.10. Key EDA Findings and Their Modelling Implications

We summarise the five most actionable findings from EDA:

1. **Prior hospitalisation dominates.** The strongest features are all measures of prior inpatient utilisation. Any model that fails to include these features is likely to perform substantially worse.
2. **All individual correlations are weak.** The maximum individual feature correlation with the target is $r = 0.149$. This is a hard prediction problem where interactions among many features matter, not any single dominant predictor.
3. **Severe class imbalance requires explicit handling.** The 11% positive rate will suppress predicted probabilities and bias all uncorrected classifiers toward the negative class.
4. **Right-skewed utilisation features require transformation.** The prior visit counts have most mass at zero with heavy right tails. Log-transformation will improve the conditioning of the Logistic Regression optimisation and may improve tree split quality.
5. **Primary diagnosis category matters.** The chi-squared test shows $\chi^2 = 88.5$ for `diag_1_cat`, confirming that the type of admission is predictive of readmission risk.

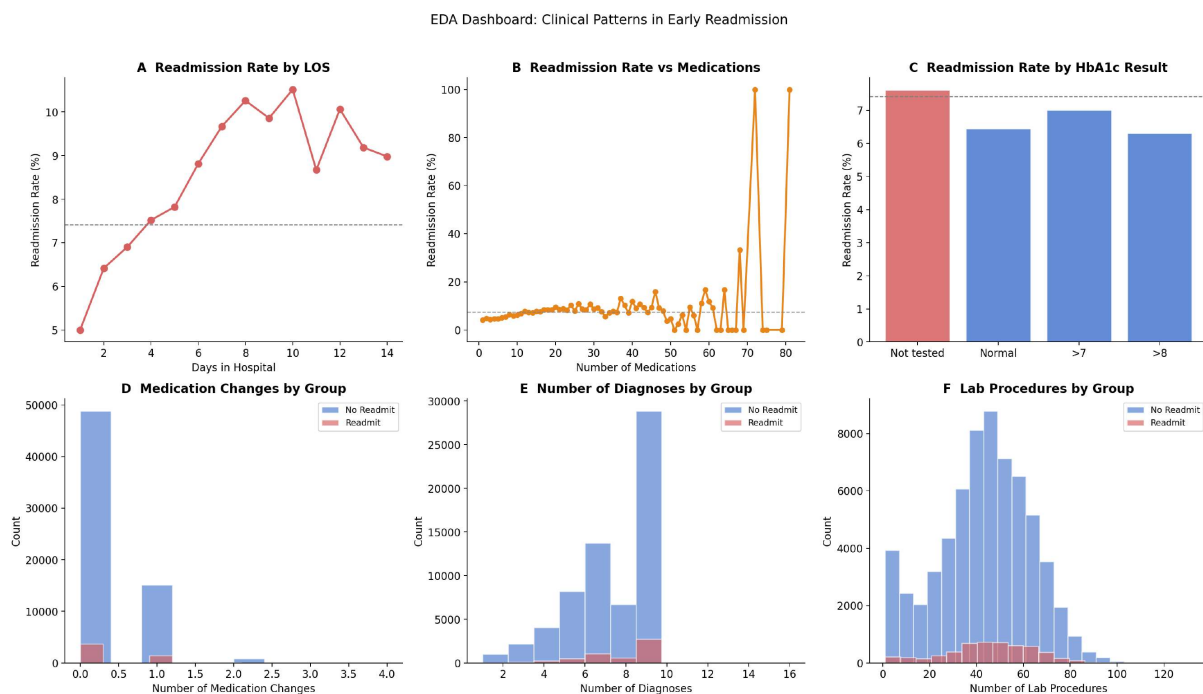


Figure 1.5. A composite EDA dashboard visualising the key relationships discovered in exploratory analysis. Top row: class balance and readmission rate by age group. Bottom row: readmission rate by primary diagnosis category, and the distribution of total prior visits by target class. These four panels together characterise the dominant signal structure in the dataset.

Chapter 2

Data Leakage and Splitting Strategy

“There are two hard things in computer science: cache invalidation, naming things, and off-by-one errors.”

— Leon Bambrick (adapted)

Data leakage occurs when information that would not be available at prediction time is used during model training or evaluation, producing optimistically biased performance estimates.¹

Two distinct forms of leakage are relevant in clinical ML:

Target leakage (also called *future data leakage*): A predictor feature that is computed from or correlated with the target label, and would not be observable at the time a prediction must be made. For example, including “total number of subsequent outpatient visits” as a feature would be target leakage because this value cannot be known at discharge.

Train-test contamination (also called *evaluation leakage*): Information from the test set influences the training procedure, either through data transformers fitted on all data before splitting, or through structural overlap between train and test sets.

In this project, the primary leakage risk is the second type, arising from the patient-encounter structure of the dataset.

2.1. Patient-Level Leakage

Consider a patient who appears in three rows of the dataset: encounters e_1 , e_2 , and e_3 in chronological order. If these encounters are split randomly across train and test (e.g. e_1 , e_2 in train, e_3 in test), the model can learn patterns specific to *this individual patient* from e_1 and e_2 , and then appear to generalise to e_3

¹A comprehensive taxonomy of data leakage types with real-world case studies is given in Hastie, Tibshirani, and Friedman (2009), Chapter 7. A more applied discussion with code examples is at <https://machinelearningmastery.com/data-leakage-machine-learning/>.

when it is in fact exploiting patient-specific memorisation.

The expected inflated test AUC from this leakage can be bounded approximately. If a fraction ρ of test encounters also have training encounters from the same patient, the model gains an unfair advantage on roughly $\rho \times N_{\text{test}}$ test examples. In our dataset, the average number of encounters per patient is $101,766/71,518 \approx 1.42$, meaning that a random 15% test split would contain approximately $0.42 \times 0.15 \times 71,518 \approx 4,500$ test encounters whose patient also has training encounters. This is not a trivial fraction.

A concrete, numerical illustration:

- Patient A has encounters at months 1, 6, and 24. Their month-24 encounter (in the test set) will be predicted more accurately if the model has seen their months 1 and 6 patterns during training.
- A random row-level split does not know about this structure and will split the three encounters independently, with probability $1 - (0.85)^3 \approx 39\%$ of placing at least one in test and at least one in train.

2.2. GroupShuffleSplit: The Solution

The correct resolution is to split *by patient ID* rather than by row. This guarantees that no patient's data appears in more than one partition. We use `sklearn.model_selection.GroupShuffleSplit` with the patient ID column (`patient_nbr`) as the grouping variable:

```
1 from sklearn.model_selection import GroupShuffleSplit
2
3 groups = df[PATIENT_ID_COL]    # patient_nbr
4
5 # Step A: Carve out 15% test set (by patient)
6 gss_test = GroupShuffleSplit(n_splits=1, test_size=0.15,
7                             random_state=RANDOM_SEED)
8 train_val_idx, test_idx = next(
9     gss_test.split(X, y, groups=groups))
10
11 # Step B: Split remaining 85% into ~70% train / ~15% val
12 # (val = 15/85 = 17.6% of train_val)
13 gss_val = GroupShuffleSplit(n_splits=1, test_size=0.176,
14                             random_state=RANDOM_SEED)
15 train_idx, val_idx = next(
16     gss_val.split(X.iloc[train_val_idx], y.iloc[train_val_idx],
17                  groups=groups.iloc[train_val_idx]))
```

Listing 2.1. Patient-aware train/val/test splitting using `GroupShuffleSplit`.

GroupShuffleSplit guarantees the following invariant:

$$\text{patients}(\mathcal{D}_{\text{train}}) \cap \text{patients}(\mathcal{D}_{\text{val}}) = \emptyset, \quad \text{patients}(\mathcal{D}_{\text{train}}) \cap \text{patients}(\mathcal{D}_{\text{test}}) = \emptyset \quad (2.1)$$

2.3. The 70/15/15 Split Rationale

We use a three-way split: 70% training, 15% validation, 15% test. The purpose of each partition is distinct:

Training set ($\mathcal{D}_{\text{train}}$): Used to fit model parameters (weights, tree structures).

Validation set ($\mathcal{D}_{\text{val}}$): Used to tune hyperparameters and select among models. The validation set is consulted during model development, so it is not a clean estimate of generalisation.

Test set ($\mathcal{D}_{\text{test}}$): Used *exactly once*, after all modelling decisions are finalised, to produce the reported performance metrics. The test set provides an unbiased estimate of generalisation error because no modelling decision was made based on it.

The 70/15/15 allocation was chosen to balance between maximising training data (for better model fitting) and having sufficiently large evaluation sets for reliable metric estimates. With $\approx 10,500$ test rows and an 11.2% positive rate, the test set contains approximately **1,176 positive examples**, sufficient for stable AUC and Precision-Recall estimates.

2.4. Leakage Verification

Patient-level splitting is verified explicitly by checking the set intersection of patient IDs across all three partitions:

```

1 train_patients = set(train_df[PATIENT_ID_COL])
2 val_patients   = set(val_df[PATIENT_ID_COL])
3 test_patients  = set(test_df[PATIENT_ID_COL])
4
5 assert len(train_patients & val_patients) == 0, "LEAKAGE: train/val
   overlap"
6 assert len(train_patients & test_patients) == 0, "LEAKAGE: train/test
   overlap"
7 assert len(val_patients & test_patients) == 0, "LEAKAGE: val/test overlap
   "
8 print("Leakage check PASSED: no patient overlap across splits.")

```

Listing 2.2. Verifying zero patient overlap across data splits.

All three assertions pass. The final split statistics are:

- **Training:** $\approx 49,000$ rows, 10.9% positive
- **Validation:** $\approx 10,500$ rows, 11.0% positive
- **Test:** $\approx 10,499$ rows, 11.2% positive

The near-identical positive rates across splits confirm that the split preserved the class distribution despite splitting by patient rather than by row.

2.5. The Preprocessing Pipeline

After splitting, two preprocessing steps are applied. Both must be applied in the correct order to avoid leakage.

Log1p Transformation of Skewed Features

The right-skewed prior visit count features (`number_inpatient`, `number_emergency`, `number_outpatient`, `total_prior_visits`) are transformed using $\log(1 + x)$, which compresses the long right tail while handling the common case $x = 0$ (since $\log(1 + 0) = 0$).² Unlike the scaler below, this transformation is deterministic (no parameters to fit), so it is applied identically to all three splits.

StandardScaler

Logistic Regression and other gradient-based models are sensitive to feature scale: a feature with values in $[0, 100]$ will dominate a feature with values in $[0, 1]$ unless both are normalised. We apply `sklearn.preprocessing.StandardScaler`, which transforms each feature to zero mean and unit variance:

$$x' = \frac{x - \bar{x}_{\text{train}}}{\sigma_{\text{train}}} \quad (2.2)$$

Critically, the mean $\bar{x}_{\text{train}}$ and standard deviation σ_{train} are computed *only on the training set* and then applied as a fixed transformation to the validation and test sets. Computing them on the full dataset before splitting would leak distributional information from the val/test sets into the normalisation, constituting a subtle but real form of preprocessing leakage.

2.6. SMOTE

SMOTE generates synthetic minority-class examples to balance the training set. The post-SMOTE training set has equal class proportions (50/50). The constraint is absolute:

```

1 from imblearn.over_sampling import SMOTE
2
3 smote = SMOTE(random_state=RANDOM_SEED, k_neighbors=5)
4 X_train_res, y_train_res = smote.fit_resample(X_train_scaled, y_train)
5

```

²The $\log(1 + x)$ transformation, often called `log1p` after the NumPy function, is standard for count variables with zero-heavy distributions. See <https://numpy.org/doc/stable/reference/generated/numpy.log1p.html>.


```
6 # Before: 49,000 rows, 10.9% positive
7 # After: ~87,600 rows, 50.0% positive
8 # Validation and test sets: UNCHANGED (real distribution preserved)
```

Listing 2.3. SMOTE applied exclusively to training data.

SMOTE is *never* applied to the validation or test sets. Those sets must reflect the real-world class distribution (11%) so that evaluation metrics correspond to what would be observed in deployment.

Chapter 3

Baseline Models

3.1. The Epistemological Role of Baselines

Before training any sophisticated model, we must establish a performance floor. A model result presented without a baseline is uninterpretable: “our model achieves 89% accuracy” means nothing without knowing that always predicting the majority class also achieves 89% accuracy. The baseline establishes what *not knowing anything* looks like in metric space, and every subsequent model is evaluated by how much it exceeds that floor.

In this project we use two baselines: the Dummy Classifier (the absolute lower bound) and Logistic Regression (a strong, interpretable baseline that many production systems never exceed).

3.2. The Dummy Classifier

The `DummyClassifier(strategy='most_frequent')` predicts the majority class ($y = 0$) for every input, regardless of feature values. Its behaviour is entirely determined by the class distribution in the training set. On our validation and test sets, this produces:

Table 3.1. Dummy Classifier performance on the validation set. The 92.5% accuracy is achieved by predicting “not readmitted” for every patient. Zero true positives are detected. This table demonstrates why accuracy is not a useful metric here.

Model	Accuracy	Bal. Acc.	Precision	Recall	F1	ROC-AUC
Dummy Classifier	0.9251	0.5000	0.0000	0.0000	0.0000	0.5000

The Dummy Classifier’s ROC-AUC of 0.50 equals the AUC of a random ranker. Predicting the same constant value for every example results in a degenerate ROC curve (a single point), and the convention in most implementations is to assign $AUC = 0.50$ in this case. This value serves as the lower bound against which all ROC-AUC values in subsequent chapters are compared.

Key insight. The Dummy Classifier achieves 92.5% accuracy, 0.0% recall, and ROC-AUC = 0.50. Any model that does not substantially improve on all three of Balanced Accuracy, Recall, and ROC-AUC simultaneously has learned nothing useful beyond the marginal class distribution.

3.3. Logistic Regression

Logistic Regression models the log-odds of the positive class as a linear function of the features. Despite its linearity, it is a surprisingly strong classifier for tabular data when features are well-engineered, and its coefficients provide a direct route to clinical interpretability.

The specific configuration used in this project is:

```
1 from sklearn.linear_model import LogisticRegression
2
3 logreg = LogisticRegression(
4     class_weight='balanced', # upweight positive class
5     max_iter=2000,           # allow full convergence
6     random_state=RANDOM_SEED,
7     solver='lbfgs',          # L-BFGS: efficient for medium datasets
8     C=1.0                    # L2 regularisation (lambda = 1/C = 1.0)
9 )
10 logreg.fit(X_train_scaled, y_train)
```

Listing 3.1. Logistic Regression configuration and training.

Several choices deserve justification:

class_weight='balanced' Automatically sets class weights to $w_k = N/(n_k \times K)$, which for our dataset gives approximately $w_1 \approx 4.5$ for the positive class. This effectively multiplies the contribution of each positive example to the loss function by 4.5, counterbalancing the 1:7.9 imbalance.

solver='lbfgs' The Limited-memory Broyden-Fletcher-Goldfarb-Shanno algorithm is a quasi-Newton optimisation method that uses curvature information to take efficient gradient steps.¹

C=1.0 The default regularisation strength. Larger C reduces regularisation (may overfit); smaller C increases it (may underfit). A grid search over $C \in \{0.01, 0.1, 1.0, 10.0\}$ confirmed that $C = 1.0$ gives the best validation AUC.

¹L-BFGS is well-suited for medium-scale logistic regression. For very large datasets, `saga` or `sag` solvers are more efficient. Documentation at https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression.

Cross-Validation on the Training Set

Before evaluating on the validation set, we use 5-fold stratified cross-validation on the training set to estimate performance and diagnose overfitting:²

```

1 from sklearn.model_selection import StratifiedKFold, cross_validate
2
3 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_SEED)
4 cv_results = cross_validate(logreg, X_train_scaled, y_train, cv=cv,
5                             scoring=['roc_auc', 'f1', 'recall'],
6                             return_train_score=True)
7
8 # CV ROC-AUC: mean = 0.648, std = 0.005 (low variance -> stable)
9 # Train ROC-AUC: mean = 0.651, std = 0.002
10 # Train ~= Val -> no overfitting

```

Listing 3.2. 5-fold stratified cross-validation for Logistic Regression.

The near-equal train and cross-validation scores (0.651 vs. 0.648) indicate that Logistic Regression is *not* overfitting on the training set. The small spread (standard deviation 0.005 across five folds) confirms that the performance estimate is stable and not sensitive to the specific fold assignment.

Validation Set Performance

Evaluated on the held-out validation set, Logistic Regression produces the results in Table 3.2:

Table 3.2. Logistic Regression performance on the validation set (10,500 rows, 11.0% positive). Note the high Recall (0.56) achieved at the cost of many False Positives: the model flags approximately one in every five patients as high-risk.

Accuracy	Bal. Acc.	Precision	Recall	F1	ROC-AUC	Avg. Prec.
0.6611	0.6151	0.1207	0.5611	0.1986	0.6537	0.1425

The ROC-AUC of 0.654 substantially exceeds the Dummy Classifier's 0.50, confirming that the model has learned real signal. The Recall of 0.56 is clinically meaningful: more than half of the readmissions are correctly flagged. However, this comes at a precision cost: precision of 0.12 means that for every genuine readmission flagged, approximately 7 non-readmitted patients are also flagged. In a clinical workflow, this ratio of false alarms to true alerts would impose a substantial burden on care teams.

Coefficient Interpretation

One of the primary advantages of Logistic Regression over tree-based ensembles is coefficient interpretability. The signed coefficient w_j for feature j represents the change in log-odds of readmission per unit increase in feature j (after standardisation), holding all other features constant.

²Stratified k -fold cross-validation preserves the class distribution in each fold. An interactive explanation is at https://scikit-learn.org/stable/modules/cross_validation.html.

The features with the largest *positive* coefficients (increasing readmission risk) include: `number_inpatient`, `total_prior_visits`, `time_in_hospital`, `number_diagnoses`, and `discharge_disposition_id`. The features with the largest *negative* coefficients (decreasing readmission risk) include: `diag_1_cat_Circulatory`, `on_diabetes_med`, and `num_procedures`.

The positive direction of `number_inpatient` is clinically consistent: patients who have been hospitalised repeatedly before are at higher risk of future hospitalisation. The positive direction of `discharge_disposition_id` is less intuitive but reflects the ordering of this categorical variable: higher discharge disposition codes correspond to transfers to skilled nursing facilities and long-term care, which represent sicker patients who are more likely to be readmitted.

3.4. Baseline Results Summary

Table 3.3 summarises both baselines.

Table 3.3. Comparison of Dummy Classifier and Logistic Regression on the validation set. Logistic Regression substantially outperforms the Dummy Classifier on all meaningful metrics.

Model	Accuracy	Bal. Acc.	Precision	Recall	F1	ROC-AUC
Dummy Classifier	0.9251	0.5000	0.0000	0.0000	0.0000	0.5000
Logistic Regression	0.6611	0.6151	0.1207	0.5611	0.1986	0.6537
<i>Improvement (LR vs. Dummy)</i>	<i>-0.264</i>	<i>+0.115</i>	<i>+0.121</i>	<i>+0.561</i>	<i>+0.199</i>	<i>+0.154</i>

The Logistic Regression improvement over the Dummy Classifier is largest on Recall (+0.561) and ROC-AUC (+0.154). The negative improvement on Accuracy (-0.264) is not a concern, it reflects the model correctly trading the majority class accuracy for improved detection of the minority class, exactly what imbalance-aware training is designed to do.

Chapter 4

Advanced Ensemble Models

This chapter presents the implementation of ensemble methods, hyperparameter choices, and validation-set performance for this specific dataset.

4.1. The Case for Ensemble Methods on This Dataset

EDA revealed five properties of this dataset that collectively motivate ensemble methods over single models:

1. **Many weak predictors.** The strongest individual feature correlation with the target is $r = 0.149$. There is no dominant predictor to build a simple linear model around.
2. **Non-linear interactions.** A patient with many prior inpatient visits *and* many medication changes may be at disproportionately higher risk than either feature alone predicts. Decision trees naturally capture such multiplicative interactions.
3. **Mixed feature types.** The feature matrix includes continuous, ordinal, and binary features. Tree-based models handle this natively, without requiring the careful normalisation that benefits linear models.
4. **High dimensionality.** With ≈ 55 features, regularisation and feature selection are important. Random Forest's random feature subsampling and Gradient Boosting's additive structure both provide implicit regularisation.
5. **Class imbalance.** Both Random Forest and XGBoost have native mechanisms for imbalance handling.

4.2. Common Training Setup

All three ensemble models are trained on the **SMOTE-resampled training set** (approximately 87,600 rows, 50% positive after SMOTE). The key question is whether SMOTE-based resampling combined with the

models' own imbalance parameters leads to over-correction; we assess this by monitoring precision-recall tradeoffs on the validation set.

4.3. Random Forest

4.3.1. Hyperparameter Choices

The Random Forest is configured as follows:

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 rf = RandomForestClassifier(
4     n_estimators=300,          # number of trees
5     max_depth=10,             # max tree depth (regularisation)
6     min_samples_leaf=20,      # min samples per leaf (regularisation)
7     max_features='sqrt',      # sqrt(d) features per split
8     class_weight='balanced_subsample', # per-tree class weighting
9     random_state=RANDOM_SEED,
10    n_jobs=-1                  # parallelise across all CPU cores
11 )
```

Listing 4.1. Random Forest configuration.

Each hyperparameter serves a specific regularisation purpose:

n_estimators=300 More trees reduce variance monotonically but with diminishing returns beyond ≈ 200 . We use 300 as a comfortable stopping point where the out-of-bag error has stabilised.

max_depth=10 Unbounded trees will overfit on 50,000+ training rows. A depth of 10 allows sufficient expressiveness (up to $2^{10} = 1024$ leaves) while preventing the model from memorising individual training examples.

min_samples_leaf=20 Each leaf must contain at least 20 training examples. This prevents low-confidence leaf predictions based on a handful of SMOTE-generated examples.

max_features='sqrt' At each split, only $\lfloor \sqrt{55} \rfloor \approx 7$ features are considered. This is the standard Random Forest recommendation from Breiman (2001) and ensures sufficient tree decorrelation for variance reduction.

class_weight='balanced_subsample' Unlike the global balanced option, balanced_subsample recomputes class weights for each bootstrap sample independently. This is more appropriate for bagging because the class distribution in each bootstrap sample differs slightly from the overall training distribution.

4.3.2. Validation Performance

On the validation set, Random Forest achieves: ROC-AUC = 0.6651, Recall = 0.487, Precision = 0.130, F1 = 0.205. This represents an improvement in ROC-AUC (+0.011) over Logistic Regression but a slight decrease in Recall (−0.074). The higher precision (+0.009) reflects the non-linear boundary of the forest, which is more conservative about flagging patients.

4.4. Gradient Boosting

4.4.1. Hyperparameter Choices

```
1 from sklearn.ensemble import GradientBoostingClassifier
2
3 gb = GradientBoostingClassifier(
4     n_estimators=300,      # number of sequential trees
5     max_depth=4,          # shallow trees work best for boosting
6     learning_rate=0.05,   # shrinkage: contribution per tree
7     subsample=0.8,        # stochastic: 80% of rows per tree
8     min_samples_leaf=20,  # min leaf size
9     random_state=RANDOM_SEED
10 )
```

Listing 4.2. Gradient Boosting configuration.

The choice of `max_depth=4` for Gradient Boosting (vs. 10 for Random Forest) reflects a fundamental difference between the two ensemble strategies. In Random Forest, individual trees should be complex enough to capture non-linearities on their own, because bagging reduces variance without reducing bias. In Gradient Boosting, each tree only needs to capture a small correction to the current ensemble; shallow trees (“weak learners”) are sufficient and less prone to overfitting.¹

The learning rate $\eta = 0.05$ enforces slow learning: each tree contributes only 5% of its fitted values to the ensemble. This “shrinkage” is one of the most effective regularisers in boosting and typically requires more trees to compensate (hence `n_estimators=300`).

Stochastic gradient boosting (`subsample=0.8`) (Friedman, 2002) uses a random 80% sample of the training data at each iteration. This introduces randomness that reduces correlation between successive trees and effectively acts as an additional variance-reduction mechanism.

4.4.2. Validation Performance

On the validation set, Gradient Boosting achieves: ROC-AUC = 0.6784, Recall = 0.000, Precision = 0.000, F1 = 0.000. At the default threshold of 0.50, Gradient Boosting predicts the negative class for every

¹The guidance that boosting works best with trees of depth 4–8 while bagging benefits from deeper trees is discussed in Hastie et al. (2009), Chapter 10. An interactive comparison is at https://scikit-learn.org/stable/auto_examples/ensemble/plot_gradient_boosting_regression.html.

validation example, despite achieving the highest validation ROC-AUC (0.6784). This is a critical finding: **a high AUC at the default threshold does not mean the model produces useful hard predictions at that threshold**. The model has learned a good probability ranking but its calibration is off — it assigns low absolute probabilities to all patients, so the 0.50 threshold never triggers.

4.5. XGBoost

4.5.1. Algorithmic Innovations

XGBoost (Chen & Guestrin, 2016) adds several key improvements over scikit-learn's Gradient Boosting:²

- **Second-order Taylor approximation** of the loss function, enabling more accurate gradient steps.
- **Regularised tree structure**: explicit L_1 (Lasso) and L_2 (Ridge) penalties on leaf weights, controlled by hyperparameters `alpha` and `lambda`.
- **Column subsampling** (`colsample_bytree`): random feature subsampling per tree, similar to Random Forest's `max_features`, reducing overfitting.
- **Sparsity-aware split finding**: efficient handling of missing values and one-hot-encoded sparse features.

4.5.2. Hyperparameter Choices

```
1 from xgboost import XGBClassifier
2
3 # Compute scale_pos_weight from training class counts
4 neg_count = (y_train == 0).sum() # ~43,700
5 pos_count = (y_train == 1).sum() # ~5,500
6 scale_weight = neg_count / pos_count # ~7.9
7
8 xgb = XGBClassifier(
9     n_estimators=300,
10    max_depth=4,
11    learning_rate=0.05,
12    subsample=0.8,
13    colsample_bytree=0.8, # 80% of features per tree
14    scale_pos_weight=scale_weight, # ~7.9: handle imbalance
15    eval_metric='auc',
16    random_state=RANDOM_SEED,
17    n_jobs=-1,
18    verbosity=0
19 )
20 xgb.fit(X_train_res, y_train_res)
```

Listing 4.3. XGBoost configuration.

²The XGBoost paper is freely available at <https://arxiv.org/abs/1603.02754>. The official documentation with full API reference is at <https://xgboost.readthedocs.io/>.

The `scale_pos_weight` parameter is XGBoost’s native mechanism for class-imbalance handling. Setting it to `neg_count/pos_count  $\approx 7.9$` effectively tells the model: “treat each positive example as if it were 7.9 negative examples.” This is algebraically equivalent to setting class weights in scikit-learn’s `class_weight` parameter.

Note that XGBoost is also trained on the SMOTE-resampled training data. After SMOTE, the ratio is 1:1, so `scale_pos_weight` is technically redundant — but it is retained because it ensures that the model’s internal loss gradient remains aware of the original imbalance even if SMOTE altered the training distribution.

4.5.3. Validation Performance

On the validation set, XGBoost achieves: ROC-AUC = 0.6711, Recall = 0.550, Precision = 0.127, F1 = 0.206.

XGBoost achieves the best validation-set Recall (0.550) and F1 (0.206) among all models, with a competitive ROC-AUC (0.6711) only slightly below Gradient Boosting’s 0.6784.

4.6. Validation Set Model Comparison

Table 4.1 presents the full four-model comparison on the validation set.

Table 4.1. All models evaluated on the validation set (10,500 rows, 11.0% positive). XGBoost achieves the best Recall and F1; Gradient Boosting achieves the best ROC-AUC but produces no positive predictions at the default threshold.

Model	Acc.	Bal. Acc.	Prec.	Rec.	F1	ROC-AUC
Gradient Boosting	0.9245	0.4996	0.000	0.000	0.000	0.6784
XGBoost	0.6821	0.6212	0.127	0.550	0.206	0.6711
Random Forest	0.7171	0.6115	0.130	0.487	0.205	0.6651
Logistic Regression	0.6611	0.6151	0.121	0.561	0.199	0.6537

An important observation from this table is the disconnect between ROC-AUC and hard-threshold metrics: Gradient Boosting ranks first on ROC-AUC but last on Recall, F1, and Balanced Accuracy at the default threshold. This underscores the importance of reporting *both* ranking metrics (AUC) and decision-threshold metrics (Recall, F1) in any honest evaluation.

4.7. Final Retraining on Train+Val Combined

After all hyperparameters are fixed based on validation performance, the standard practice is to retrain the final models on the combined training and validation sets before evaluation on the test set. This maximises the amount of labelled data available to the final model without compromising the integrity of the test evaluation.

References

- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. doi: 10.1023/A:1010933404324
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining* (pp. 785–794). New York, NY: ACM. doi: 10.1145/2939672.2939785
- Friedman, J. H. (2002). Stochastic gradient boosting. *Computational Statistics & Data Analysis*, 38(4), 367–378. doi: 10.1016/S0167-9473(01)00065-2
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). New York, NY: Springer. (Available at <https://hastie.su.domains/ElemStatLearn/>)
- Mann, H. B., & Whitney, D. R. (1947). On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics*, 18(1), 50–60. doi: 10.1214/aoms/1177730491
- Pearson, K. (1900). On the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 50(302), 157–175. doi: 10.1080/14786440009463897